Synégo

ARCHITECTURE · INSTALLATION · EXPLOITATION

Documentation *technique*

Pour l'équipe qui développe, déploie et maintient le service : architecture, code, base de données, configuration, chaîne CI/CD, installation d'un serveur, exploitation et procédures d'urgence.

DÉPÔT	github.com/luidgi-gaudin/projetkine · branche main
PRODUCTION	VPS Linode 172.239.9.200 · https://172-239-9-200.sslip.io
PILE	.NET 10 · React 19 · PostgreSQL 17 · nginx · Docker Compose
VERSION	4 novembre 2026 · commit f827ae3 en production

Sommaire

01	Vue d'ensemble	pile technique, organisation du dépôt
02	Architecture	schéma, conteneurs, flux
03	Réseau et sécurité	ports, TLS, en-têtes, secrets, rôles
04	API .NET	authentification, routes, services de fond
05	Base de données	comptes, tables, migrations
06	Front-end React	structure, routes, i18n, thème, Firebase
07	Configuration	référence du fichier .env
08	Développer en local	stack de dev, stack de prod locale
09	Intégration et déploiement continus	GitHub Actions, déploiement par pull
10	Installer un serveur	install.sh, durcissement, cohabitation Zabbix
11	Exploitation	commandes, tunnels, sauvegardes, rotation de clé
12	Procédures d'urgence	repérer, traiter, dire
13	Qualité et limites connues	tests, contrôles, reste à faire

01

Vue d'ensemble

Synégo est un monorepo : une API ASP.NET Core, un front React servi par nginx, et l'infrastructure (Docker Compose, scripts serveur, CI) versionnée au même endroit. Tout tourne en conteneurs, en développement comme en production.

Pile technique

COUCHE	TECHNOLOGIE	RÔLE
API	.NET 10, Minimal API, EF Core 10 + Npgsql 10.0.3	Logique métier, contrôle d'accès, intégrations
Authentification	Firebase Auth (@firebase/auth 1.13.9) + Jwt-Bearer 10.0.14	E-mail, SMS, Google ; l'API vérifie les jetons
Front	React 19.3, React Router 8.5, TypeScript 6.0, Vite 8.3	Site vitrine, espace client, back-office
Animations	motion 14.3 (`motion/react`), Lenis 1.3	Animations, défilement doux (coupés si mouvement réduit)
Base de données	PostgreSQL 17 (alpine)	Schéma `synego`, comptes séparés
Web	nginx 1.27 (alpine)	TLS, CSP, limite de débit, proxy `/api`, fichiers statiques
E-mails	MailKit 4.18 (SMTP), Mailpit en démo	Rappels de rendez-vous 48 h / 24 h
Rendez-vous	Calendly API v2 + webhooks	Réservation, annulation, lien Meet
Stockage documents	Disque du serveur (défaut) ou Firebase Storage	Fichiers clients, servis uniquement par l'API
Exploitation	Docker Compose, systemd, certbot 3.1, Uptime Kuma 1.23, Adminer 4.8	Déploiement, certificats, surveillance
CI	GitHub Actions	Build, tests, migrations, audit, démarrage de la stack

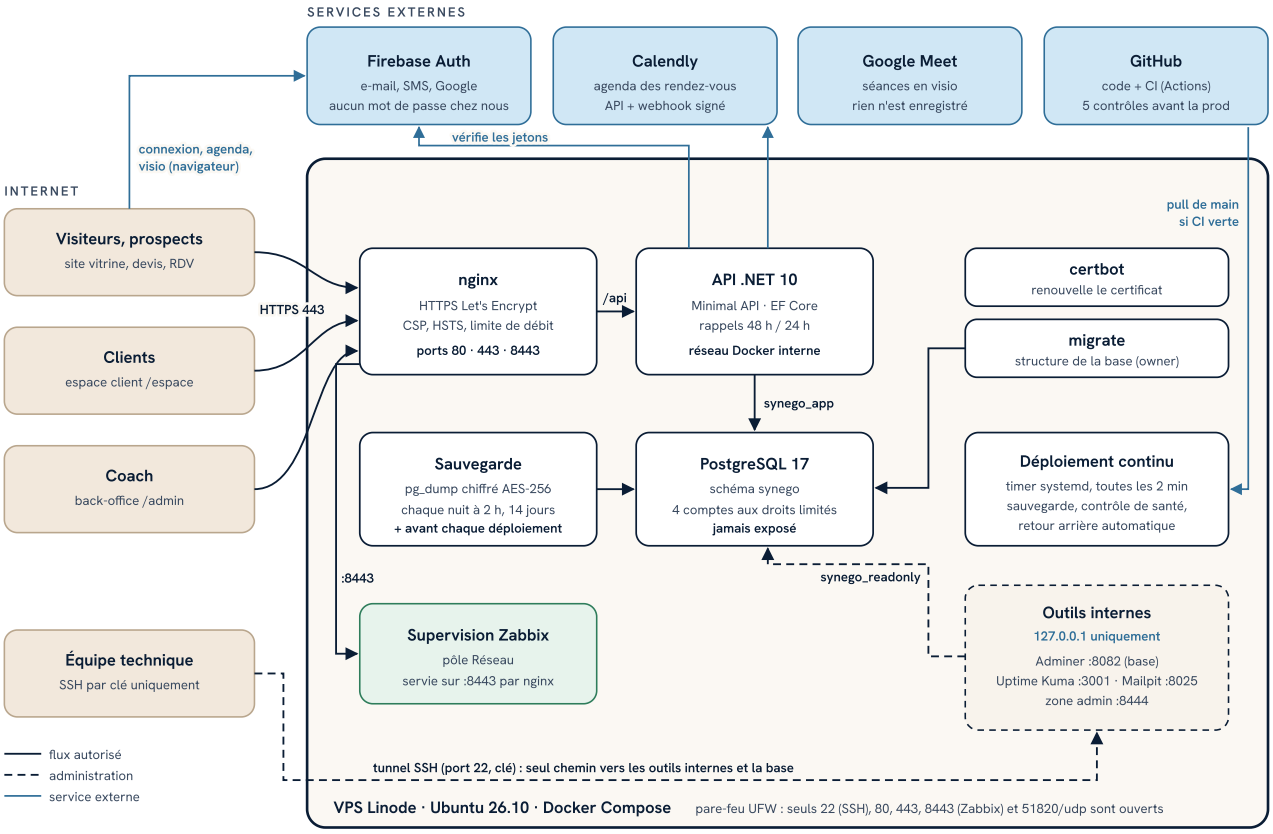
Organisation du dépôt

CHEMIN	CONTENU
`Synégo.Server/`	API : `Authentication`, `Booking` (Calendly), `Data` (entités, migrations), `Documents`, `Endpoints`, `Notifications`, `Security`, `Program.cs`
`Synego.Client/`	Front : `src/pages` (site, auth, espace, admin), `src/components`, `src/i18n`, `src/theme`, `src/auth`, `src/styles`
`tests/Synego.Server.Tests/`	Tests xUnit de l'API

CHEMIN	CONTENU
`compose.yaml`, `compose.-dev.yaml`	Stack de production, surcouche de développement
`synego.sh`	Lanceur Docker (init du `.env`, contournement des chemins accentués)
`deploy/api`, `deploy/nginx`, `deploy/backup`, `deploy/postgres`	Dockerfiles, gabarits nginx, scripts de sauvegarde, rôles PostgreSQL
`deploy/vps/`	`install.sh`, `synego-deploy.sh` + unités systemd, durcissement, WireGuard
`.github/workflows/ci.yml`	Pipeline d'intégration continue
`docs/`	Architecture, déploiement, kit Scrum (backlog, urgences, démo), livrables PDF
`scripts/trello_import.py`	Création du tableau Trello à partir de `docs/scrum/backlog.json`

02

Architecture



Architecture en production (4 novembre 2026).

Conteneurs (projet Compose `synego`)

SERVICE	IMAGE	RÔLE	PORTS PUBLIÉS
`nginx`	`synego-nginx` (build)	Front compilé, TLS, en-têtes, proxy `/api`, zone admin	80, 443 ; `ADMIN_BIND_IP:ADMIN_PORT` → 8081
`api`	`synego-api` (build)	API .NET, services de fond (rappels, purge)	aucun (réseau Docker)
`migrate`	`synego-api`	Applique les migrations EF avec `synego_owner`, puis s'arrête	aucun
`db`	`postgres:17-alpine`	Base de données, rôles créés au premier démarrage	aucun
`backup`	`synego-backup` (build)	pg_dump chiffré selon `BACKUP_CRON`, `restore.sh`	aucun
`certbot`	`certbot/certbot:v3.1.0`	`certbot renew` toutes les 12 h (webroot)	aucun
`adminer`	`adminer:4.8.1`	Consultation de la base	`ADMIN_BIND_IP:8082`

SERVICE	IMAGE	RÔLE	PORTS PUBLIÉS
`mailpit`	`axllent/mailpit:v1.27`	Boîte e-mail de démonstration (SMTP par défaut)	`ADMIN_BIND_IP:8025`
`uptime-kuma`	`louislam/uptime-kuma:1.23.16`	Surveillance et alertes	`ADMIN_BIND_IP:3001`

Volumes : `pgdata` (base), `appdata` (documents en mode Local et clés Data Protection), `letsencrypt`, `certbot-www`, `uptime-kuma` ; dossier `./backups` pour les sauvegardes. En production, `ADMIN_BIND_IP=127.0.0.1`.

Parcours d'une requête

1. Le navigateur charge le front depuis nginx (HTTPS). Il se connecte directement à Firebase Auth, qui lui remet un jeton d'identité.
2. Chaque appel `/api/…` porte ce jeton (`Authorization: Bearer`). nginx limite le débit puis relaie vers l'API.
3. L'API vérifie la signature du jeton auprès de Google (`securetoken.google.com/<projet>`), retrouve ou crée le compte en base, et lit le **rôle en base** (jamais dans le jeton).
4. Les rendez-vous sont pris dans l'iframe Calendly ; le front confirme la réservation à l'API, qui la relit chez Calendly (et le webhook la synchronise sur une offre payante).

03

Réseau et sécurité

Ports et pare-feu

PORT	ÉCOUTE	SERVICE	ACCÈS
22/tcp	hôte	SSH	Internet (UFW), équipe uniquement
80, 443/tcp	0.0.0.0	nginx (site, API, ACME)	Internet
8443/tcp	0.0.0.0	nginx → `zabbix-web` (surcouche locale)	Internet, compte Zabbix
51820/udp	hôte	WireGuard (prêt, non activé)	Internet, clé obligatoire
8444, 8082, 3001, 8025	127.0.0.1	zone admin, Adminer, Uptime Kuma, Mailpit	tunnel SSH uniquement
5432, 8080	réseau Docker	PostgreSQL, API	jamais exposés

> **DOCKER CONTOURNE UFW**
>
> Un port publié par Docker est ouvert même si UFW le refuse. Règle du projet : ne publier que 80, 443 (et 8443 pour Zabbix) sur toutes les interfaces ; tout autre port est lié à `${ADMIN_BIND_IP}` dans `compose.yaml`.

TLS et en-têtes

- Certificat Let's Encrypt obtenu par `install.sh` (webroot), renouvelé par le conteneur `certbot` ; nginx recharge les certificats toutes les 12 h (`synego-reload-certs`). Sans certificat, un auto-signé temporaire permet de démarrer.
- `EXTRA_DOMAINS` ajoute d'autres noms au même certificat (production : `synego.duckdns.org`).
- En-têtes : CSP stricte (sources : soi-même, Google pour l'authentification et reCAPTCHA, Calendly), HSTS, `X-Frame-Options`, `nosniff`. `font-src 'self'` : aucune police en `data:` (Vite configuré pour ne jamais les intégrer).
- Limite de débit : 10 req/s par IP sur `/api` (rafale 40) côté nginx ; 10 envois par 10 minutes et par IP sur les formulaires publics côté API.

Authentification et rôles

- JwtBearer valide l'émetteur, l'audience (`Firebase__ProjectId`) et la signature des jetons Firebase.
- Un `IClaimsTransformation` (`UserProvisioning`) crée le compte applicatif à la première requête et ajoute le rôle lu en base : **Prospect**, **Client** (signé) ou **Coach**.
- Les e-mails de `Admin__Emails` deviennent Coach uniquement si l'adresse est **vérifiée** par Firebase. Un coach peut en nommer d'autres (invitations) mais pas se retirer lui-même.
- Back-office : politique « Coach ». Option `ADMIN_VPN_ONLY=true` : l'API exige en plus l'en-tête `X-Synego-Zone: admin`, posé uniquement par le bloc nginx de la zone admin (le bloc public l'écrase).

Secrets

SECRET	OÙ IL VIT
Mots de passe PostgreSQL, phrase de sauvegarde	`.env` du serveur (généré aléatoirement par `synego.sh init`, `chmod 600`)
Clé web Firebase (`VITE_FIREBASE_API_KEY`)	`.env` uniquement, jamais dans le code ; restreinte dans Google Cloud aux sites du service et aux API Identity Toolkit et Token Service
Jeton Calendly de la coach	En base, chiffré par ASP.NET Data Protection (clés dans le volume `appdata`)
Clé de service Firebase (mode Storage Firebase)	`secrets/*.json`, ignoré par git, monté en lecture seule
Accès SSH	Clés publiques dans `/root/.ssh/authorized_keys`

Contrôles : `.env`, `secrets/*.json` et `compose.override.yaml` sont ignorés par git. Le scan de secrets de GitHub est actif sur le dépôt public.

Fichiers et données

- Dépôt de documents : PDF, PNG, JPEG, DOCX, XLSX, PPTX ; 10 Mo maximum ; extension *et* signature binaire vérifiées ; nom de stockage aléatoire ; téléchargement uniquement par l'API après contrôle du propriétaire.
- Suppression d'un document : douce (`DeletedAt`), purge définitive après 30 jours. Un client ne supprime que ses propres dépôts.
- Calendly : l'API ne suit que des URI `api.calendly.com` (pas de SSRF) ; webhook signé HMAC, accepté pendant 3 minutes.

04

API .NET

Routes

GROUPE	ROUTES	ACCÈS
`/api/health`, `/api/ping`	santé (base + stockage, 503 si l'un ne répond pas), vivacité	public
`/api/contact`	`POST` demande de devis ou de contact (crée un ticket), limité en débit, champ piège	public
`/api/booking`	`GET /types` ; `POST /confirm` ; `GET /mine` ; `POST /mine/refresh` ; `POST /{id}/cancel` (refus à moins de 24 h)	public / connecté
`/api/webhooks/calendly`	`POST` événements Calendly signés	signature HMAC
`/api/me`	`GET` profil ; `DELETE` suppression du compte	connecté
`/api/documents`	`GET` ; `POST` dépôt ; `GET /{id}/file` ; `DELETE /{id}`	connecté
`/api/tickets`	`GET` ; `POST` ; `GET /{id}` ; `POST /{id}/messages`	connecté
`/api/admin`	`GET /overview` ; clients (`GET`, `PATCH /{id}`, documents, notes) ; `GET/PATCH /appointments`, `POST /appointments/{id}/cancel` ; tickets (`GET`, `PATCH`, messages) ; `/document-categories`	Coach
`/api/admin/calendly`	`GET` état ; `POST /connect` ; `DELETE` ; `PUT /types` ; `POST /sync`	Coach
`/api/admin/coaches`	`GET` ; `POST` nommer ; `DELETE /{id}` ; `DELETE /invitations`	Coach

Services de fond

- **Rappels** (`ReminderService`) : toutes les 5 minutes (`IntervalMinutes`), envoie le rappel 48 h (bouton d'annulation) aux rendez-vous entre 24 et 48 h, puis le rappel 24 h (lien Meet, plus d'annulation). Un rendez-vous pris tard ne reçoit que les rappels encore utiles. Les envois sont marqués en base (`Reminder48SentAt`, `Reminder24SentAt`) : jamais de doublon.
- **Purge des documents** (`DocumentService`) : efface définitivement les fichiers supprimés depuis plus de 30 jours.

Intégration Calendly

- La coach colle un jeton personnel dans le back-office ; l'API le vérifie, le chiffre, puis liste ses types d'événements.
- Chaque type du site (`Booking:Types`) est associé à un événement Calendly, masquable ; webhook automatique sur offre payante.
- Réservation : l'iframe envoie `calendly.event_scheduled` ; le front appelle `/api/booking/confirm`, l'API relit l'invité chez Calendly (idempotent sur l'URI) et récupère le lien de visio (`location.join_url`).

05

Base de données

Comptes et droits

COMPTE	UTILISÉ PAR	DROITS
`postgres`	personne (initialisation)	superutilisateur
`synego_owner`	`migrate`, restauration	propriétaire du schéma `synego`
`synego_app`	l'API	`SELECT/INSERT/UPDATE/DELETE` sur les tables, rien sur la structure
`synego_backup`	`backup`	lecture seule
`synego_readonly`	l'équipe via Adminer	lecture seule

Posés par `deploy/postgres/init/01-roles.sh` au premier démarrage du volume, avec `ALTER DEFAULT PRIVILEGES` pour les nouvelles tables. Connexions sans GSSAPI (`GSS Encryption Mode=Disable`).

Tables (schéma `synego`)

TABLE	CONTENU
`user_accounts`	Compte applicatif lié à l'identité Firebase (`FirebaseUid` unique), rôle, date de signature
`appointments`	Rendez-vous recopiés de Calendly, statut, lien Meet, traces des rappels, langue
`client_documents`	Métadonnées des documents (fichier sous un nom aléatoire), suppression douce
`tickets`, `ticket_messages`	Demandes de devis, de contact et de support, avec leur fil
`client_notes`	Notes privées de la coach (épinglables), jamais exposées au client
`app_settings`	Réglages du back-office (compte Calendly, jeton chiffré, correspondance des événements)
`__ef_migrations`	Historique des migrations EF Core

Migrations

```
# créer une migration après modification d'une entité (depuis la racine)
dotnet ef migrations add NomDeLaMigration --project "Synégo.Server"
# vérifier qu'aucune migration ne manque (fait aussi par la CI)
dotnet ef migrations has-pending-model-changes --project "Synégo.Server"
```

Les migrations s'appliquent automatiquement au démarrage de la stack (service `migrate`, avant l'API).

06

Front-end React

Structure

DOSSIER	CONTENU
`src/pages/site`	Accueil, Séminaires, Accompagnement, Approche, Rendez-vous, Contact, pages légales
`src/pages/auth`	Connexion (e-mail, téléphone, Google), inscription
`src/pages/espace`	Espace client : accueil, rendez-vous, documents, demandes, profil
`src/pages/admin`	Back-office : aujourd'hui, clients, rendez-vous, demandes, Calendly, coachs
`src/components`	Mise en page, marque (cadran, logo), réservation, mouvement, interface (dialogues, notifications)
`src/i18n`	`fr.ts` (référence), `en.ts` (même structure, vérifiée par TypeScript), `brand.ts` (coordonnées)
`src/auth`	Configuration Firebase, fournisseur d'authentification, traduction des erreurs
`src/theme`	Thème clair / sombre (`data-theme`), préférence mémorisée

Points d'implémentation

- **Routes** : site public, `/connexion`, `/inscription`, `/espace/*` (connecté), `/admin/*` (Coach). Les pages sont chargées à la demande (`lazy`) ; le back-office interroge `/api/admin/overview` toutes les 60 s pour ses pastilles.
- **Langues** : dictionnaire typé, langue mémorisée (`synego:locale`) ; ajouter une langue = copier `en.ts` et la déclarer.
- **Thème** : suit le système, mémorisé (`synego:theme`) ; couleurs en variables CSS dans `index.css`.
- **Mouvement** : `MotionConfig reducedMotion="user"` ; Lenis désactivé si l'utilisateur réduit les animations.
- **Polices** auto-hébergées (Fraunces, Hanken Grotesk) : aucune requête vers Google Fonts.
- **Dialogues** à la marque (`confirm`, `ask`, `toast`) à la place des fenêtres natives du navigateur.

Configuration Firebase côté navigateur

`src/auth/firebase.ts` lit `VITE_FIREBASE_*` depuis le `.env` (lu par Vite en développement, passé en `build args` à l'image nginx en production). **La clé web n'a aucune valeur de repli dans le code** : si elle manque, le site s'affiche et la connexion échoue avec un message clair (`auth/invalid-api-key`). Les autres identifiants (projet, domaine d'authentification, identifiant d'application) gardent un défaut public.

`VITE_FIREBASE_PHONE_TEST_MODE=true` désactive la vérification d'application pour les numéros de test déclarés dans la console Firebase.

07

Configuration

Un seul fichier : `.env` à la racine, créé par `./synego.sh init` depuis `.env.example` (mots de passe aléatoires). Il n'est jamais versionné.

VARIABLE	RÔLE
`DOMAIN`	Nom public du service (certificat, liens des e-mails, `App__PublicUrl`)
`HTTP_PORT`, `HTTPS_PORT`	Ports publics de nginx (80, 443)
`ADMIN_BIND_IP`, `ADMIN_PORT`	Interface et port des outils internes (production : `127.0.0.1`, 8444)
`ADMIN_VPN_ONLY`	`true` : back-office accessible uniquement par la zone admin
`Firebase__ProjectId`	Projet Firebase dont l'API accepte les jetons
`Admin__Emails`	E-mail(s) de la coach principale (Coach si vérifié)
`VITE_FIREBASE_API_KEY`	**Obligatoire** : clé web Firebase (uniquement ici)
`VITE_FIREBASE_*` (5 autres)	Domaine d'authentification, projet, bucket, expéditeur, identifiant d'application
`Storage__Provider`, `Storage__Bucket`, `FIREBASE_CREDENTIALS_NAME`	Stockage des documents : `Local` (défaut) ou `Firebase`
`CALENDLY_URL_*`, `CALENDLY_TOKEN`, `CALENDLY_WEBHOOK_KEY`	Liens par défaut et accès Calendly (normalement réglés depuis le back-office)
`SMTP_HOST`, `SMTP_PORT`, `SMTP_USERNAME`, `SMTP_PASSWORD`, `SMTP_SECURITY`	Serveur d'envoi des rappels (défaut : Mailpit)
`EMAIL_FROM`, `EMAIL_REPLY_TO`	Expéditeur et adresse de réponse des e-mails
`POSTGRES_PASSWORD`, `SYNEGO_OWNER_PASSWORD`, `SYNEGO_APP_PASSWORD`, `SYNEGO_BACKUP_PASSWORD`, `SYNEGO_READONLY_PASSWORD`	Mots de passe des comptes PostgreSQL (générés)
`BACKUP_PASSPHRASE`, `BACKUP_CRON`, `BACKUP_RETENTION_DAYS`, `BACKUP_DIR`	Chiffrement, horaire (défaut `0 2 * * *`), rétention (14 jours), dossier des sauvegardes

SURCOUCHE PROPRE AU SERVEUR

`compose.override.yaml` (non versionné, chargé automatiquement par Docker Compose) ajoute en production le port 8443, le bloc nginx `deploy/nginx/zabbix.conf` et le réseau externe de Zabbix. Il est généré par `install.sh` quand un conteneur `zabbix-web` est détecté.

08

Développer en local

Prérequis

Docker Desktop ; pour travailler hors conteneurs : .NET SDK 10 et Node 24. Le dossier du projet contient un accent (« projetkiné ») : passez toujours par `./synego.sh`, qui contourne un défaut de Docker Compose avec les chemins accentués.

Stack de développement (rechargement à chaud)

```
./synego.sh init                # une fois : crée .env (renseigner VITE_FIREBASE_API_KEY, Admin__Emails)
./synego.sh dev up              # front Vite (HMR) + API dotnet watch + base + Mailpit
```

ADRESSE	SERVICE
`http://localhost:5173`	Front Vite, rechargement instantané
`http://localhost:8035`	Mailpit de développement (rappels e-mail)

Stack de production en local

```
./synego.sh up -d --build       # identique à la production, sur https://localhost
./synego.sh logs -f api         # journaux d'un service
./synego.sh down                # arrêt (ajouter -v pour effacer les volumes)
```

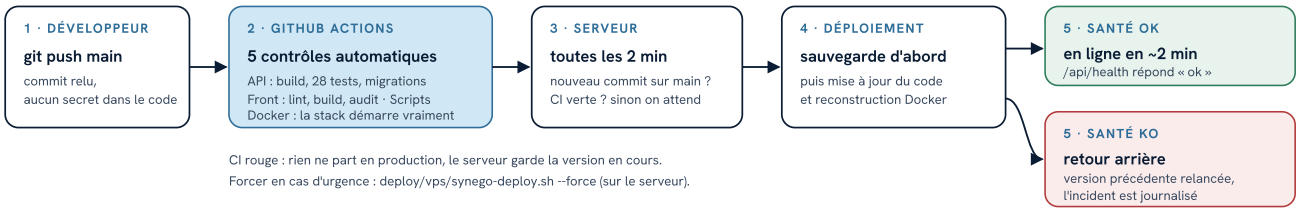
Avec `DOMAIN=localhost` et `ADMIN_BIND_IP=127.0.0.1` : site sur `https://localhost` (certificat auto-signé), zone admin sur `http://localhost:8443`, Adminer `:8082`, Uptime Kuma `:3001`, Mailpit `:8025`.

Contrôles avant de pousser

```
dotnet build -c Release && dotnet test tests/Synego.Server.Tests
npm --prefix Synego.Client run lint && npm --prefix Synego.Client run build
```

09

Intégration et déploiement continus



De la modification du code à la mise en production, sans intervention.

GitHub Actions (`.github/workflows/ci.yml`)

JOB	CONTRÔLES
API .NET	Restauration, build Release (avertissements = erreurs), 28 tests xUnit, migrations à jour (`has-pending-model-changes`), paquets vulnérables
Front React	`npm ci`, lint, types et build, `npm audit --omit=dev --audit-level=high`
Scripts shell	`shellcheck -S warning` sur tous les scripts
Docker	Construit les images, démarre toute la stack, vérifie `/api/health` et les codes attendus (200, 401, 404, 301)
Prêt pour la production	Sur `main` uniquement, quand tout est vert : c'est ce statut que le serveur attend

Déploiement par pull (`deploy/vps/synego-deploy.sh`)

1. `synego-deploy.timer` lance le service toutes les 2 minutes (et 2 minutes après le démarrage du serveur). Un verrou empêche deux déploiements simultanés.
2. `git fetch` : si `origin/main` n'a pas bougé, rien à faire.
3. État de la CI du commit lu par l'API publique de GitHub (`check-runs`) : en cours → on attend ; échec → on ne déploie pas.
4. Sauvegarde de la base (`backup.sh`), puis `git checkout` du commit et `docker compose up -d --build --remove-orphans`.
5. Contrôle de `https://localhost/api/health` : succès → commit écrit dans `.deployed`, anciennes images nettoyées ; échec → **retour au commit précédent** et reconstruction.

```
journalctl -u synego-deploy -f                    # suivre les déploiements
systemctl list-timers synego-deploy.timer         # prochain passage
/opt/synego/deploy/vps/synego-deploy.sh --force   # déployer sans attendre la CI (urgence)
```

Option : `GITHUB_TOKEN` dans `/etc/synego-deploy.env` si le dépôt devient privé.

10

Installer un serveur

Un serveur Ubuntu ou Debian neuf, en root, une commande. Le script est rejouable sans risque.

```
curl -fsSL https://raw.githubusercontent.com/luidgi-gaudin/projetkine/main/deploy/vps/install.sh \
  | ADMIN_EMAILS=coach@exemple.fr FIREBASE_API_KEY=<clé web> bash
```

Ce que fait `install.sh`

1. Paquets, Docker et Compose v2, swap de 2 Go si moins de 3 Go de RAM ; pare-feu UFW (22, 80, 443, 51820/udp).
2. Clone de `main` dans `/opt/synego`. Un dossier existant sans git est **mis de côté** (rien n'est supprimé) ; un `.env` étranger est renommé et remplacé.
3. `.env` : mots de passe aléatoires, `DOMAIN` (défaut `<ip>.sslip.io`), `ADMIN_BIND_IP=127.0.0.1`, projet Firebase, coach, clé web.
4. Zabbix détecté : `ADMIN_PORT=8444` et génération de `compose.override.yaml`.
5. Ports 80/443 libérés si un autre conteneur ou service les occupe (arrêté, pas supprimé), puis construction et démarrage.
6. Certificat Let's Encrypt pour `DOMAIN` et `EXTRA_DOMAINS`, rechargement de nginx.
7. Installation et activation de `synego-deploy.service` et `synego-deploy.timer`.

VARIABLE	EFFET
`ADMIN_EMAILS`	Coach principale
`FIREBASE_API_KEY`	Écrit `VITE_FIREBASE_API_KEY` dans `.env` (avertissement si absente)
`DOMAIN`, `EXTRA_DOMAINS`	Nom principal, noms supplémentaires du certificat
`LETSENCRYPT_EMAIL`, `FIREBASE_PROJECT_ID`	Avis d'expiration Let's Encrypt ; autre projet Firebase que celui du dépôt

Durcissement et VPN (facultatif)

```
DEPLOY_USER=deploy bash deploy/vps/01-hardening.sh   # utilisateur dédié, SSH par clé, fail2ban, mises à jour auto
bash deploy/vps/02-wireguard.sh                      # serveur WireGuard 10.8.0.1
bash deploy/vps/wg-add-peer.sh gerante 10.8.0.2      # profil + QR code
```

Avec le VPN actif : `ADMIN_BIND_IP=10.8.0.1` dans `.env`, puis réserver SSH au VPN (`ufw allow in on wg0 to any port 22 proto tcp && ufw delete allow 22/tcp`).

Nouveau domaine : l'ajouter dans Firebase › Authentication › Domaines autorisés, et `https://<domaine>/*` aux sites autorisés de la clé « Synego web » (Google Cloud).

11

Exploitation

Commandes courantes (sur le serveur, dans `/opt/synego`)

COMMANDE	USAGE
`docker compose ps`	État des conteneurs
`docker compose logs --tail 100 api nginx db`	Journaux récents
`docker compose up -d`	Relancer ce qui est arrêté
`docker compose exec backup backup.sh`	Sauvegarde immédiate
`docker compose stop api && docker compose exec backup restore.sh && docker compose start api`	Restaurer la dernière sauvegarde
`docker compose run --rm --entrypoint certbot certbot renew`	Renouveler le certificat à la main
`docker compose exec nginx synego-reload-certs`	Recharger les certificats
`curl -s https://localhost/api/health -k`	Santé de l'API

Accéder aux outils internes par tunnel SSH

```
ssh -N -L 8082:127.0.0.1:8082 -L 3001:127.0.0.1:3001 -L 8025:127.0.0.1:8025 -L 8444:127.0.0.1:8444
root@172.239.9.200
# puis, sur le poste : http://localhost:8082 (Adminer, compte synego_readonly), :3001 (Uptime
Kuma),
# :8025 (Mailpit), :8444 (zone admin)
```

Surveillance

- `/api/health` répond `{"status":"ok","database":true,"documents":true}`, ou 503 dès que la base ou le stockage ne répond plus.
- Uptime Kuma : moniteur « HTTP(s) – mot-clé » sur `https://DOMAINE/api/health` (mot-clé `"ok"`, 60 s), moniteur de certificat (alerte à 14 jours), notification e-mail, Telegram ou Discord.
- Zabbix (pôle Réseau) : supervision de l'hôte, interface sur le port 8443.

Sauvegardes et test de restauration

Chaque nuit à 2 h et avant chaque déploiement : `pg_dump` du schéma `synego` au format custom, chiffré AES-256 (`BACKUP_PASSPHRASE`), rotation à 14 jours, dans `/opt/synego/backups`. **À ajouter** : une copie hors du VPS.

Tester sans toucher à la production : restaurer dans une base temporaire.

```
docker compose exec -T db psql -U postgres -c "create database synego_restore_test owner syne-
go_owner"
docker compose exec -T db psql -U postgres -d synego_restore_test -c "create schema synego autho-
rization synego_owner"
docker compose exec -T backup sh -c 'f=$(ls -1t /backups/synego_*.dump.enc | head -n1);
  openssl enc -d -aes-256-cbc -pbkdf2 -pass env:BACKUP_PASSPHRASE -in "$f" -out /tmp/t.dump &&
  PGUSER=synego_owner PGPASSWORD="$SYNEGO_OWNER_PASSWORD" pg_restore --no-owner --schema=synego -d
synego_restore_test /tmp/t.dump'
# comparer les comptages avec la base réelle, puis :
docker compose exec -T db psql -U postgres -c "drop database synego_restore_test"
```

Test réalisé le 4 novembre 2026 : restauration, effacement volontaire d'un compte dans la copie, nouvelle restauration, compte retrouvé.

Changer la clé web Firebase

1. Google Cloud › Identifiants : créer une clé restreinte (sites du service, API Identity Toolkit et Token Service).
2. Écrire la nouvelle valeur dans `VITE_FIREBASE_API_KEY` du `.env` (serveur et postes).
3. Reconstruire nginx (`docker compose up -d --build nginx`, ou attendre le prochain déploiement).
4. Vérifier que le site embarque la nouvelle clé, puis supprimer l'ancienne dans Google Cloud.

12

Procédures d'urgence

SITUATION	REPÉRER	TRAITER	DIRE AU CLIENT
Site hors ligne	Alerte de surveillance, `/api/health` en erreur	`docker compose ps`, journaux, `docker compose up -d` ; si le dernier déploiement est en cause : `git checkout` du commit de `.deployed` précédent et reconstruction	« Le site a été indisponible X minutes ; nous avons été prévenus et l'avons relancé. »
Donnée effacée	Signalement	Restauration (chapitre 11) ; un document supprimé reste sur le disque 30 jours	« Nous avons récupéré l'information grâce à la sauvegarde de cette nuit. »
Compte compromis	Signalement, activité anormale	Console Firebase : désactiver l'utilisateur ; back-office : repasser en Prospect ; retirer un coach si besoin	« Nous avons bloqué l'accès ; vos données sont intactes. »
Secret exposé	Alerte GitHub, usage anormal	Restreindre, remplacer (chapitre 11), supprimer l'ancien, clore l'alerte en « révoqué »	« Une clé a été changée par précaution ; l'ancienne ne fonctionne plus. »
Attaque, trafic anormal	Réponses 429, journaux nginx	`ufw insert 1 deny from <IP>` ; les limites de débit protègent déjà les formulaires	« Les protections ont bloqué la tentative, aucune donnée n'a été touchée. »
Certificat expiré	Moniteur de certificat	`certbot renew` puis `synego-reload-certs`	« Le cadenas de sécurité est rétabli. »
Disque plein	`df -h`	`du -sh backups`, baisser `BACKUP_RETENTION_DAYS`, `docker system prune`	—
Calendly ou Firebase en panne	Erreurs côté site	Le site reste en ligne ; renvoyer vers la page Contact (les demandes deviennent des tickets)	« Un service partenaire a eu une panne ; vos clients pouvaient toujours vous écrire. »

13

Qualité et limites connues

Tests et contrôles

- **28 tests xUnit** : règles des documents (formats, signatures, tailles), Calendly (URI autorisées, signature des webhooks, correspondances), e-mails de rappel (annulation proposée à 48 h, lien Meet et annulation fermée à 24 h, traduction anglaise, échappement HTML du nom).
- Build .NET en `TreatWarningsAsErrors` ; ESLint et TypeScript strict côté front ; structure des traductions vérifiée à la compilation.
- Audits de dépendances (NuGet vulnérables, `npm audit`), `shellcheck`, démarrage réel de la stack dans la CI.

Limites connues et suite

POINT	ACTION PROPOSÉE
Alerte de surveillance non encore configurée	Moniteur Uptime Kuma ou déclencheur Zabbix sur `/api/health`, notification testée
VPN WireGuard prêt mais non activé	`02-wireguard.sh`, puis `ADMIN_BIND_IP=10.8.0.1` ; aujourd'hui, tunnel SSH
SSH accepte encore les mots de passe	`01-hardening.sh` (clé uniquement, fail2ban) et changement du mot de passe root
Sauvegardes sur le même VPS	Copie quotidienne vers un stockage externe
Pas de SMTP réel en production	Compte SMTP (par exemple Brevo), SPF et DKIM du domaine
Calendly non relié en production	La coach colle son jeton depuis le back-office, durées alignées sur les textes
Adresse `sslip.io` bloquée par certains filtres (« Dynamic DNS »)	Nom de domaine dédié
Deux tâches planifiées du pôle Réseau visent l'ancien dossier	Les retirer (le service a ses propres sauvegarde et renouvellement)
Domaine `synego.duckdns.org` non autorisé dans Firebase	L'ajouter (Firebase et restriction de la clé) si ce domaine doit servir à la connexion